

AI Usage Declaration

Course: CS202 — Programming Techniques **Project:** Apple Knight Adventure (2D action-platformer, C++17 / raylib / LDtk) **Repository:** [CS202-FinalProject](#) **Period covered:** 2026-06-08 → 2026-08-31

About this document

Our team used an AI coding assistant during development. We did not keep verbatim chat transcripts for the whole project, so **this log is a reconstruction**: each entry was rebuilt from the commit that the conversation produced, and records the substance of what we asked and what we got rather than the exact wording. The commit hashes and messages referenced in each entry are real and can be checked against [git log](#).

Everything below was reviewed, compiled, and tested by us before being committed. Where the assistant produced something wrong, that is recorded too — several entries are follow-up prompts after a first answer failed.

Division of work. The project was split by layer. Nguyễn Anh Kiệt owned the View/UI/rendering layer, the animation system, the boss system, the LDtk and Map Builder tooling, and the audio system. Nguyễn Trọng Tiến owned the Model layer and entity hierarchy, level design and map authoring, gameplay systems (checkpoints, save data, elemental combat, buffs), and gameplay bug-fixing.

Member	Primary area	Commits
Nguyễn Anh Kiệt	View / rendering, UI, animation, boss system, tooling, audio	~158
Nguyễn Trọng Tiến	Model, level design, gameplay systems, save data, bug-fixing	~42

Part 1 — Nguyễn Anh Kiệt

K-01 · Animation system and texture atlas

Date: 2026-06-10 **Related commits:** [Add AnimationSystem with Animator class and functionality for clip management](#), [Add TextureAtlas class for managing textures and animation clips from JSON](#), [Refactor AnimationSystem and TextureAtlas for improved memory management and JSON parsing](#)

Prompt

I am writing a 2D game in C++17 with raylib. I need an animation system with two classes: [TextureAtlas](#) that loads a spritesheet plus a JSON descriptor (Aseprite-style [frames](#) map with [x/y/w/h](#), and an optional [clips](#) map with a frame-name list and per-frame durations), and [Animator](#) that plays a named clip and reports the current source rectangle. Requirements: several animators must be able to share one atlas without reloading the texture, and a clip must be able to loop or play once. Show me the headers and the parsing code.

AI response (summary) Produced `TextureAtlas` with a `frames` name→`Rectangle` map and a `clips` name→`AnimationClip` map, plus `Animator` holding a pointer to the current clip, a frame index, and an accumulator ticked in `Update(dt)`. Atlas sharing was handled with `std::shared_ptr<TextureAtlas>` handed out by a cache keyed on the JSON path.

Outcome Accepted as the base of `View/Animator.*` and `View/TextureAtlas.*`. We changed two things: the assistant unloaded the raylib `Texture2D` in the destructor without checking `id != 0`, which crashed when a JSON path was wrong and the texture had never loaded; and clip durations were assumed uniform, which did not fit our hand-timed boss frames, so we changed `durations` to a per-frame vector.

K-02 · Render command queue and layer sorting

Date: 2026-06-11 → 2026-06-12 **Related commits:** `feat: implement Renderer class with initialization, shutdown, and sprite submission functionality`, `feat: add RenderTypes header with Layer enum and RenderCommand struct`, `Systems: renderer core fixes + lock-free MPSC queue + ResizeWindow + dropped counter race fix`

Prompt

Our entities currently call `DrawTexture` directly from `Render()` inside the Model classes, so draw order depends on entity iteration order and the Model depends on raylib. I want a `Renderer` singleton that entities submit draw commands to (`SubmitSprite(texture, src, pos, scale, rotation, origin, tint, layer, z, flipX, entityId)`), which then sorts by layer then z and flushes once per frame. Explain the trade-offs of sorting per frame vs keeping buckets.

AI response (summary) Recommended fixed per-layer buckets (`Background`, `World`, `UI`) with a preallocated command array per bucket and an index array sorted with `std::sort` on flush, so the command structs themselves are never moved. Warned that submitting from a non-main thread needs a queue and gave an MPSC ring buffer sketch.

Outcome Accepted. This became `View/Renderer.cpp` and let us delete every `Render()` method from the Model layer (commit `cleanup(model): remove all Render() traces from Model layer`). The threading part we kept but simplified — we only submit from the main thread in practice, and the queue is a guard rail. The assistant's first version of the dropped-command counter used a plain `int` incremented from both paths; we changed it to `std::atomic` after a data race showed up under `-fsanitize=thread`.

K-03 · View bug sweep after the render refactor

Date: 2026-06-13 **Related commits:** `fix: various View bugs – UIManager menu layer, missing includes, EnemyStatusRenderer layer/world->screen, SkillSlotData operator==, GameView missing Update call, audio init guards, entity ID assertion, stale comments`

Prompt

After moving all rendering into the View layer, several things broke at once: the menu draws underneath the HUD, enemy status icons are drawn at world coordinates so they slide off-screen when the camera moves, and `GameView` never animates because nothing ticks the animators. Here are the

relevant files. Go through them and list what is actually wrong, most severe first, instead of rewriting everything.

AI response (summary) Reported seven distinct defects: `UIStateManager` pushed the menu onto the world layer instead of the UI layer; `EnemyStatusRenderer` passed world coordinates to a screen-space draw and needed `GetWorldToScreen2D` with the active camera; `GameView::Render` never called `EntityRenderer::Update(dt)`; `SkillsSlotData` lacked `operator==` so the "slot changed" check never fired; two missing includes that only compiled by luck through transitive headers; `SoundManager` calls before `InitAudioDevice`; and an entity-ID assertion that fired on the valid ID 0.

Outcome All seven were real and all were fixed in that commit. This was the single most useful AI interaction of the project — asking for a ranked diagnosis rather than a rewrite kept the diff small and reviewable.

K-04 · Boss phase state machine

Date: 2026-08-04 → 2026-08-05 **Related commits:** Refactor boss implementation and add new boss types, Enhance boss mechanics with combo steps, super armor, and damage tracking, Enhance boss mechanics and projectile behavior: add 2D scaling, refine phase transitions, and implement new projectile subtypes

Prompt

I have one `Boss` base class and three subclasses. Each boss has 3–4 phases that change at health thresholds; a phase change plays a transition animation during which the boss must be invulnerable and must not act; and inside a phase the boss runs multi-step melee combos where each step has its own hit window. Design the state machine. I want phase data to be declarative per subclass, not a `switch` repeated in three files.

AI response (summary) Suggested a `BossState` enum (`Idle`, `Move`, `Attack`, `Hurt`, `Transition`, `Die`) held in the base class, with each subclass supplying a table of phase descriptors (health threshold, move set, cooldowns). Combos were modelled as a step index plus a per-step active window, and "super armor" as a flag that suppresses the `Hurt` transition while still applying damage.

Outcome Adopted as the shape of `Boss.*`, `Boss1/2/3`. We rejected the assistant's proposal to drive phase transitions from inside `TakeDamage` — a transition that starts in the middle of collision resolution left projectiles referencing a boss that had already swapped its hitbox. We moved the check to the boss's own `Update`, which is why `BossState::Transition` is tested explicitly in the projectile collision code.

K-05 · Oversized animation frames breaking hitbox alignment

Date: 2026-08-04 → 2026-08-05 **Related commits:** Add scale multiplier to `AnimationClip` and normalize oversized frames in `CharacterRenderer`, Implement hit tracking for projectiles and adjust boss cooldowns and scaling in `CharacterRenderer`

Prompt

Boss hurt and ultimate animations come from sheets with much larger frames than the idle sheet (the artist padded them for the effect). When those clips play, the boss visually doubles in size and no

longer lines up with its collision box. I do not want to re-export the art. How do I normalize this in the renderer?

AI response (summary) First answer: scale every frame so its width matches the idle frame width. That was wrong for us — the padding is not symmetric, so width-matching made the boss sink into the floor.

Second prompt from us clarified that the sprites are anchored at the feet. The assistant then proposed a per-clip `scale` multiplier stored in `AnimationClip` plus a per-clip ground-origin point, so the renderer anchors on the feet and scales around that anchor.

Outcome The second answer was correct and shipped as `GetCurrentClipScale()` and `GetCurrentGroundOrigin()` in `Animator`, consumed by `CharacterRenderer::RenderAll`. Recorded here because the first answer was accepted, tested, and then reverted.

K-06 · LDtk level loader

Date: 2026-07-24 → 2026-07-26 **Related commits:** `feat: integrate LDtk level editor (full MVC)`, `fix: compile errors with nlohmann json and stat64i32 on MinGW 14`, `fix: adjust LDtk entity position scaling for correct rendering`

Prompt

We want to author maps in LDtk instead of a custom text format. Given a `.ldtk` JSON file, write a loader that reads: level pixel size, the `Collision` `IntGrid` layer into a solid map, the `Tiles/BG_Tiles` tile layers, and the `Entities` layer where each entity identifier maps to one of our game classes. Use `nlohmann/json`. Assume the game renders at 64px tiles while LDtk authors at 16px.

AI response (summary) Produced the pass structure still in `LevelFactory::LoadLDtkLevel`: parse tileset defs into a `UID→tileType` map, pass 1 builds the solid set from `intGridCsv`, pass 2 walks the layers and dispatches entities by `__identifier`.

Outcome Accepted, but it took three rounds to actually work:

1. `layer["__tilesetDefUid"]` is JSON `null` on `Entities` and `IntGrid` layers, and `nlohmann::json::value()` throws `type_error.302` when a key exists but is null — it only falls back for *absent* keys. The assistant's code used `value()` and threw on every map. Fixed with an explicit null check.
2. `intGridCsv` can contain `null` for empty cells; `get<int>()` threw. Fixed with a null guard.
3. Entity positions were written straight from `px`, so every entity landed at 1/4 of its correct position. The assistant had applied the 64/16 scale to tiles but not to entities. This is the `fix: adjust LDtk entity position scaling` commit.

The MinGW `stat64i32` link error was unrelated to the AI — it came from an older raylib binary against MinGW 14 and we solved it ourselves in `CMakeLists.txt`.

K-07 · Minimap with fog-of-war exploration

Date: 2026-08-21 **Related commits:** `feat: Implement minimap functionality with exploration tracking and rendering`

Prompt

Add a minimap that only reveals areas the player has visited. Our maps are up to 258×217 tiles, so I do not want a per-tile visited array drawn every frame. Suggest a representation and a draw strategy, and tell me where in the frame it should be updated.

AI response (summary) Recommended coarse exploration cells (a block of N×N tiles marked explored when the player centre enters it) stored in a hash set, with the map itself drawn from the already-loaded tile list transformed to panel space. Entities of interest (checkpoints, portals, cup, boss) drawn as shape markers, filtered by whether their cell is explored.

Outcome Accepted and implemented as `View/MinimapView.cpp`. The marker shapes are ours — diamond for checkpoint, ring for portal, pentagon for the level-complete cup, circle for boss.

K-08 · Audio manifest and sound pooling

Date: 2026-08-14 → 2026-08-25 **Related commits:** `Add new sound assets and update audio manifest, feat: rebuild gameplay sound effects`

Prompt

Sounds are currently loaded ad-hoc with hardcoded paths and the same effect retriggering every frame produces a machine-gun sound. Design a manifest-driven `SoundManager`: a JSON file maps a logical event name to one or more samples with volume, a pitch range, and a minimum cooldown, and `PlaySound("enemy_hurt")` picks a sample, randomizes pitch, and refuses to play if the cooldown has not elapsed.

AI response (summary) Gave the manifest schema we now use in `assets/sounds/audio_manifest.json` (`samples`, `volume`, `pitch: [min,max]`, `cooldown`) and a `SoundManager` that preloads samples once and keeps a last-played timestamp per event.

Outcome Accepted essentially as given. The cooldown field is what fixed the footstep and `enemy_hurt` spam.

K-09 · Level source adapter refactor

Date: 2026-08-31 **Related commits:** `Add level source adapter and remove unused runtime code`

Prompt

`LevelFactory::LoadLevel` decides between the legacy `.lvl` text format and the LDtk format with a string suffix comparison in the middle of the function, and the two loaders are now ~700 lines in one file. Refactor to an interface so the format choice is a lookup, without changing behaviour — the legacy loader must stay the fallback for extensionless paths and extension matching must become case-insensitive.

AI response (summary) Proposed an `ILevelSourceAdapter` with `CanLoad(path) / Load(request)` and a `LevelLoadRequest` struct bundling the four parameters, with `LegacyLevelAdapter` and `LDtkLevelAdapter` as the two implementations and the legacy one as the default.

Outcome Accepted and applied — this is the current shape of `Factories/LevelSourceAdapter.h` and the top of `LevelFactory.cpp`. We deliberately kept it as a pure move with no behaviour change so the refactor could be verified by playing every level.

Part 2 — Nguyễn Trọng Tiến

T-01 · Entity and Character hierarchy

Date: 2026-06-10 **Related commits:** `Foundation and Model hierarchy`, `Build system + include paths`

Prompt

I am designing the Model layer for a 2D platformer. I need a base `Entity` (id, position, size, velocity, rotation, scale, active flag, type tag, virtual `Update`) and a `Character : Entity` adding health, damage, direction, and an attack timer. Entity types include Player, Enemy, Boss, Item, Chest, Checkpoint, Projectile, TeleportPortal, FakeWall. Advise on whether to use an enum tag plus `static_cast` or `dynamic_cast` for type dispatch, given this is a course project graded on OOP.

AI response (summary) Recommended an `EntityType` enum tag with `static_cast` at the dispatch sites — RTTI lookups in a per-frame loop over hundreds of entities are measurably slower, and the enum also serializes directly into the save format. Suggested keeping virtual `Update` for polymorphic behaviour so the design still demonstrates inheritance.

Outcome Adopted. This is the `EntityType / GetType()` pattern used throughout the codebase.

T-02 · Character state auto-inference

Date: 2026-06-12 **Related commits:** `add Character::State enum with auto-inference for Player/Enemy`

Prompt

The View needs to know which animation clip to play, but the Model should not know about clips. I want `Character` to expose a `State` (Idle, Run, Jump, Fall, Attack, Hurt, Die) that is derived from physics and timers rather than set by hand at every call site, so nobody can forget to update it.

AI response (summary) Suggested computing the state on read from velocity, ground flag, and the attack / hurt / death timers, with a documented precedence order (death > hurt > attack > airborne > moving > idle) so two conditions can never disagree.

Outcome Adopted. The precedence order is what stopped the "attacking while falling" flicker we had before.

T-03 · Teleport portals and boss-arena transitions

Date: 2026-08-03 → 2026-08-05 **Related commits:** `Create Map 1 and Portal to transition to Boss Area`, `Create world2.ldtk`, `Implement entity merging and enhance teleport portal alignment`

in GameState and LevelFactory

Prompt

I need two kinds of portal. A *local* portal teleports the player to its partner in the same level, paired by a colour id set in LDtk. A *boss arena* portal moves the player into a separate arena room and back afterwards, restoring their state. Portals are authored in LDtk with a `PortalType`, `ColorId` and `TargetLevelId` field. How should the pairing and the state save/restore work?

AI response (summary) Recommended a two-stage load: create all portals first, then run a pairing pass that links portals sharing a `ColorId`, since a portal's partner may be parsed after it. For the arena, snapshot the player's health, max health, score, skill points, inventory and cores into a struct before the transition and restore afterwards, restoring max health before current health so a `SetHealth` clamp cannot silently truncate a buffed pool.

Outcome Adopted. The max-health-before-health ordering detail was correct and non-obvious — we had exactly that bug with the VitalCore before applying it, and the comment explaining it is still in `RestorePlayerState`.

T-04 · FakeWall secret passages

Date: 2026-08-09 **Related commits:** `feat: implement full FakeWall logic with damage, collision, and hints`

Prompt

I want breakable fake walls that look like ordinary terrain. Constraint: a wall is an LDtk entity placed on top of a normal tile, it must block the player exactly like a solid tile does, and it must disappear after taking enough damage. Walk me through what has to change in collision, damage, and rendering.

AI response (summary) Proposed a `FakeWall` entity with its own small health pool, resolved in a second collision pass alongside the tile pass so it blocks movement identically; damage routed through the existing melee hit-box lambda; and on destruction, deactivating the entity *and* removing the underlying tile from the Main layer so the passage actually opens.

Outcome Adopted and shipped. Note for the record: the assistant only wired damage into the melee path, and neither of us noticed at the time that ranged-only characters could therefore never open these walls. That gap was found and fixed much later — see T-09.

T-05 · Elemental reaction system

Date: 2026-08-29 → 2026-08-30 **Related commits:** `Modify Magae Character and Boss Logic, Apply element system for other character`

Prompt

I want an elemental system. Four elements (Fire, Water, Thunder, Void) each apply a status (Burn, Wet, Shocked, Corroded) on hit. Hitting a target that already carries a status with a *different* element triggers

a reaction with a damage multiplier and possibly a new status. Design the data layout so designers can tune it without touching combat code, and so adding a fifth element later is cheap.

AI response (summary) Recommended two flat tables: an element profile table (element → status, aura duration, colour) and a reaction table keyed on (existing status, incoming element) producing a multiplier, a resulting status and its duration. Combat code then does a single lookup and stays element-agnostic.

Outcome Adopted as `Systems/ElementalSystem.cpp`. We then asked a follow-up:

The Magic Caster's skills each carry a fixed element, but the other three classes are physical. I do not want to hand-author elements per skill for every class.

The assistant suggested that a projectile with no explicit element inherit whatever infusion buff the player is currently running, so a Ninja shuriken under Infuse Fire sets up the same reactions a Fireball does. That is the behaviour in `SpawnPlayerProjectile` today and it is why the element system reaches all four classes with almost no per-class code.

T-06 · Save data and level unlocking

Date: 2026-08-31 **Related commits:** `Update SaveManager.cpp`, `Edit Static Saving and Update Checkpoint Trigger`

Prompt

`SaveManager` writes a single `save.json` holding player name, coins, unlocked characters, per-level high score / best stars / best time, achievements, lifetime stats and leaderboards. Two problems: a partially corrupted file currently throws and wipes everything, and level unlocking reads the save in a way that lets a hand-edited file expose later levels. Suggest fixes.

AI response (summary) For loading, parse each section defensively — wrap each per-key conversion in its own try/catch and skip only the malformed entry rather than aborting the whole load. For unlocking, do not trust a set of "unlocked" ids from the file; derive unlock state by walking levels in order and stopping at the first one with no recorded score and no stars, so a non-sequential save cannot skip ahead.

Outcome Both adopted. The sequential-walk unlock is the loop in `MenuController` and the per-key guard is the `readIntMap` lambda in `SaveManager::Load`.

T-07 · Merge conflict: two designs for finishing a level

Date: 2026-08-31 **Related commits:** `Fix Crash game error`, `Edit map`, merge commit on branch `Tien`

Prompt

I have a merge conflict in `GameController.cpp` and `GameState.cpp`. Both sides implement the same feature — pressing F at the end-of-level checkpoint should complete the level without killing every enemy — but differently. One side adds a `m_levelCompleteByPlayer` flag; the other removes the old enemy-count check and has `IsLevelComplete()` read the checkpoint's activated state. Do not just pick one; work out which is actually correct and resolve it.

AI response (summary) Compared both against the rest of the code and found that the flag-based side never calls `Activate()` on the endgame checkpoint, while `UpdateEndgameCheckpoints()` drives the finish-flag animation entirely from `IsActive()` — so that side would have shipped a finish flag that never animates. Recommended taking the activation-based logic as the base, keeping the flag (its declaration had already merged cleanly into the header) wired in as a second trigger, and dropping four variables on the flag side that the merged loop no longer read.

Outcome Resolved as recommended. Verified by rebuilding and checking the finish animation still plays.

T-08 · Checkpoint placement audit and the level-complete trophy

Date: 2026-08-31 **Related commits:** `Update render and hitbox`, `Edit map`, `Update lv12.ldtk`

Prompt

Check every map for checkpoints that are placed wrong or render wrong, and fix the positions. Then make the end-of-level checkpoint in every map a golden trophy like the one in the tutorial. Read the asset and `.ldtk` files to do it.

AI response (summary) Rather than eyeballing it, the assistant wrote throwaway tools that parse the `.ldtk` files with the project's own `nlohmann/json` and re-render each map offline using the game's exact tile draw rules, then reported: the project's convention is a prop at grid row `gy` with the solid tile at `gy+1`; all 60 mid checkpoints and all 5 endgame checkpoints in the playable levels already satisfied it; the tutorial's `Checkpoint_Statue` was authored half a tile high and floated 32px; and the trophy renderer anchored its art to the *top* of the collision box, leaving the pedestal hanging in the air in the tutorial too.

Outcome Three changes taken: `CheckpointEnd` now builds a `LevelCompleteCup` at load time, dropped so its base lands on the tile the flag stood on (`MakeGroundedCup` in `LevelFactory.cpp`); `TutorialRenderer` now anchors the trophy to the bottom of its box; and the tutorial statue was moved down one row in `tutorial.ldtk`. The assistant also flagged that this makes the endgame-checkpoint code path unreachable — we chose to keep it, since the `.lv1` map-builder format can still produce one.

T-09 · Bug: invisible, frozen player after Second Wind

Date: 2026-08-31 **Related commits:** fix applied to `GameController::TryRevive`

Prompt

In level 3, while I was fighting near a checkpoint and a core appeared, my character suddenly went invisible and stopped moving. Skills still fire, I can still see the projectiles, but the character does not move and is not drawn anywhere. Why?

AI response (summary) Traced it to the Second Wind core rather than to the checkpoint. `Character::TakeDamage` clears the entity's `m_active` flag when health reaches zero. The normal death path `RespawnPlayer` calls `SetActive(true)`; `TryRevive` — the Second Wind path — restored health but never restored the flag. The three symptoms then follow from three systems checking different flags: `CharacterRenderer` skips `IsActive()` so the player is invisible, `Character::Update` returns early on `!m_active` so the position never integrates, but `HandlePlayerInput` guards on `IsAlive()` (health > 0, now

true) so input still fires skills. The "core appearing" was the floating text that `TryRevive` itself emits with the core's name.

Outcome Fixed with one line — `player->SetActive(true)` in `TryRevive` — and the assistant checked the other health-restoring paths (potions, the Ghost pet, `RestorePlayerState`) for the same missing invariant; none had it.

T-10 · Magic Caster fireball breaking fake walls

Date: 2026-08-31 **Related commits:** change applied to `GameController::UpdatePlayerProjectiles`

Prompt

Make the Magic Caster's fire skill able to break fake walls.

AI response (summary) Found that fake walls only ever took damage inside the melee hit-box lambda, and the Magic Caster's whole kit is projectiles, so that class could never open a secret at all (see T-04). Added a fake-wall pass to the player-projectile update, gated on the projectile carrying `DamageType::Fire`, and placed it **before** the solid-tile sweep — the level 3 fake wall sits on a solid collision tile, so checking after would kill the shot on the exact tile it is meant to clear.

Outcome Applied. The assistant noted the consequence that a Ninja shuriken under Infuse Fire also breaks walls, since projectiles inherit the active infusion (T-05), and left the choice to us; we kept it, as it is consistent with the elemental design.

Summary of AI usage

Category	How AI was used
Architecture / design advice	Animation system, render queue, boss state machine, elemental tables, entity hierarchy, level source adapter
Code generation	Atlas and JSON parsing, LDtk loader skeleton, minimap, sound manifest
Debugging	View bug sweep (K-03), LDtk null handling (K-06), Second Wind revive (T-09), merge conflict analysis (T-07)
Analysis tooling	Throwaway offline map renderers used to audit checkpoint placement (T-08)
Not used for	Level design and map authoring, art and audio asset selection, gameplay balance numbers, the report documents

What we did not accept. Several AI suggestions were tested and rejected: width-based frame normalization (K-05), phase transitions driven from inside `TakeDamage` (K-04), and `nlohmann::json::value()` used on nullable keys (K-06). Every entry above was compiled and play-tested before being committed, and the team members named on each part are responsible for the code in them.